A Pure Python Neural Network Library for Educational and Research Purposes: Architecture, Implementation, and MNIST Case Study

Mikołaj Mocek[1]

[1]*Politechnika Śląska, Gliwice, Poland*

Abstract

This paper presents a neural network library implemented from scratch in pure Python and intended primarily for education, experimentation, and lightweight research prototyping. The project exposes the internal structure of model definition, forward propagation, backpropagation, and weight updates, while still supporting practical components such as dense layers, convolution, pooling, flattening, serialization, and dataset abstractions. The implementation is validated on the MNIST handwritten digit dataset. In addition to describing the software architecture, the paper documents a new parser for the official IDX image and label formats, a lightweight convolutional experiment, and a discussion of the current limitations of the framework. The resulting system is not positioned as a high-performance alternative to industrial frameworks, but rather as a transparent environment for understanding how neural networks operate internally.

Keywords

neural networks, pure Python, deep learning, MNIST, educational software, convolutional neural networks

1. Introduction

Deep learning systems are now embedded in a broad spectrum of technical and scientific workflows, from image classification and medical signal interpretation to speech interfaces, recommendation engines, and document understanding. Their practical success is strongly connected with the availability of mature software ecosystems. Frameworks such as TensorFlow [1] and PyTorch [2] allow researchers to assemble sophisticated models with relatively little code, while high-level APIs such as Keras simplify rapid prototyping and classroom demonstrations. At the same time, this convenience creates a well-known educational problem: when most of the important operations are hidden inside highly optimized kernels, students often learn to use neural networks before they learn to reason about them. A model may converge, diverge, or silently produce poor predictions without the learner fully understanding which part of the computation graph or training loop is responsible.

This issue is especially visible when introducing the mathematical core of neural computation. A multilayer perceptron depends on the interplay between affine transformations, nonlinear activation functions, and gradient propagation [3, 4, 5]. Convolutional neural networks (CNNs) extend this idea by learning local spatial filters and hierarchical feature maps that are particularly useful in vision tasks [6, 7, 8, 9, 10]. Recurrent neural networks and Long Short-Term Memory networks address temporal dependencies by explicitly modeling sequential state updates [11], whereas transformer architectures replace recurrence with attention and have become a dominant paradigm for language and multimodal processing [12]. These architectural families differ not only in topology but also in the shape of tensors they process, the training instabilities they exhibit, and the regularization or initialization strategies they require. Techniques such as Xavier initialization [5], adaptive optimization with Adam [13], batch normalization [14], and rectifier-aware initialization [15] were introduced precisely because training deep models is not a trivial extension of shallow learning.

For this reason, low-level educational implementations remain useful even in the presence of industrial-grade libraries. A pure Python codebase offers three advantages. First, it makes the com-

IVUS 2025: International Conference Information Society and University Studies, 2025

mikolaj.mocek@icloud.com (M. Mocek)

putation path auditable: a student can inspect how weights are stored, how batches are generated, and how each derivative is propagated across layers. Second, it provides a controlled environment for experimentation. When a developer writes a new convolutional layer, data parser, or optimizer manually, correctness issues become immediately visible and can be checked against the expected tensor shapes and numerical values. Third, such a codebase creates a bridge between theory and production software. Concepts introduced abstractly in textbooks, such as filter banks, pooling operators, or the chain rule, become concrete when represented as ordinary Python classes and lists.

The library presented in this paper was designed in exactly this spirit. Its goal is not to compete with optimized tensor runtimes, but to support transparent study of neural-network internals and to serve as a minimal research sandbox. The implemented modules cover dense layers, activation and loss functions, 2D convolution, max pooling, flattening, serialization, and dataset abstractions. During the work reported here, the project was extended with an explicit parser for MNIST binary IDX files, compatibility improvements in the convolution and pooling stack, and an executable CNN experiment using the architecture "Conv2D $\rightarrow$ Conv2D $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense(10)". The paper therefore makes two contributions. The first is software-oriented: it documents the organization of a small but extensible pure Python framework. The second is empirical: it demonstrates that the framework can load MNIST-compatible data, execute convolutional training end to end, and produce reproducible figures for analysis, even if the current implementation is still far from the accuracy and efficiency of established frameworks.

2. Dataset

The study is based on the MNIST dataset of handwritten digits [6]. The original dataset contains 70,000 grayscale images of size 28×28, divided into 60,000 training samples and 10,000 test samples. Each example belongs to one of ten classes representing digits from 0 to 9. In the official distribution, images are stored in the binary `idx3-ubyte` format, while labels are stored in `idx1-ubyte`. To support direct use of the original benchmark, the library now includes a dedicated `MnistDataset` class and low-level parsers for both binary formats. The implementation validates magic numbers, decodes image headers, reconstructs 2D pixel matrices, and supports one-hot encoding of labels.

Two practical data-access modes are supported. In eager mode, all decoded images are loaded into memory as Python lists. In lazy mode, only metadata and labels are retained, while individual images are read on demand during batch generation. This design reduces unnecessary RAM pressure and is more appropriate for an educational pure Python framework, where the cost of duplicating large nested lists is relatively high. The dataset layer therefore serves both a pedagogical role and a software-engineering role: it shows how binary formats can be parsed manually and how a training loop can be decoupled from storage details.

The repository snapshot used for the experiment documented in this draft did not include raw IDX files, but it did include a PNG conversion of MNIST. For that reason, the executed convolutional sanity-check used a balanced subset of the PNG version derived from the same benchmark content: 80 training samples and 40 test samples, with class-balanced sampling. The important point is that the library now supports the official binary format directly, while the figures and metrics shown below were produced from the locally available PNG representation of the same handwritten-digit domain. All images were normalized to the $[0, 1]$ range before training.

3. Proposed Solution

The proposed solution is a modular pure Python library that keeps the computational path explicit and easy to inspect. The framework provides the following core capabilities:

- explicit dataset abstractions, including batching, shuffling, and lazy loading,
- dense, convolutional, pooling, and flatten layers,

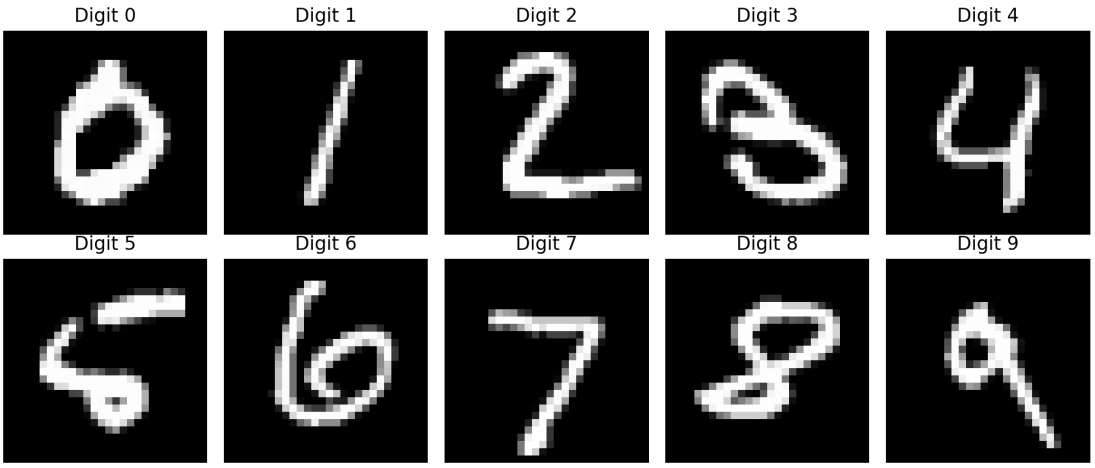

Figure 1: Representative MNIST samples used in the experiment, one image per digit class.

- pluggable activation, loss, and learning-rate functions,
- a sequential training model with forward and backward propagation,
- serialization helpers for saving model states, and
- experiment scripts that generate evaluation figures directly from the repository.

4. Library Design and Implementation

4.1. Dataset Handling and IDX Parsing

The base `Dataset` abstraction stores inputs, outputs, batch size, and randomization state. During this work, its shuffling procedure was corrected so that batches follow the actual randomized index order rather than the original sequence. The new `MnistDataset` specialization extends this logic with direct support for `idx3-ubyte` and `idx1-ubyte`. The parser reads the binary header using `struct.unpack`, verifies the expected MNIST magic numbers, reconstructs each image row by row, and optionally normalizes pixel values. The lazy variant reads image bytes only when the batch generator requests them, which avoids eagerly duplicating the full dataset in memory.

Example usage of the parser is shown below:

```
from nns.core.datasets.mnist_dataset import MnistDataset

train_dataset = MnistDataset(
    imagesFilePath="train-images-idx3-ubyte",
    labelsFilePath="train-labels-idx1-ubyte",
    batchSize=32,
    normalize=True,
    oneHot=True,
    lazy=True
)
```

4.2. Layer Abstraction

All trainable and non-trainable operations are represented as Python layer classes. The library currently includes `Dense`, `Convolution2D`, `MaxPooling2D`, and `Flatten`. The `Convolution2D` layer was updated to support both legacy fixed-kernel tests and trainable multi-channel kernels. It now handles single-channel matrices, multi-channel feature maps, and batched inputs more consistently. The pooling

layer stores max indices from the forward pass so that gradients can be routed back only through the winning locations during backpropagation. The flatten layer records the input shape and reconstructs it during the backward pass.

4.3. Functions, Initialization, and Training

Activation functions and losses are implemented as explicit classes with `call` and `derivative` methods. In the current experiment, the final dense classifier uses a linear activation and mean squared error. This is not an optimal objective for digit classification, but it matches the intentionally simple structure of the framework. Xavier initialization [5] is used for dense weights, while convolution kernels are initialized from small uniform values. The training loop is coordinated by the `Sequential` model, which propagates values forward through the layer stack and then walks backward through the same stack to accumulate gradients and update trainable parameters.

4.4. Convolutional Experiment

To validate the convolution path requested in this stage of the project, a dedicated non-interactive script was added. The script trains a compact CNN with two convolutional layers, one max-pooling layer, one flatten operation, and a dense output layer with ten neurons:

```
model = Sequential([
    Convolution2D(out_channels=2, kernel_size=(3, 3), seed=1),
    Convolution2D(out_channels=2, kernel_size=(3, 3), seed=2),
    MaxPooling2D(poolSize=(2, 2)),
    Flatten(),
    Dense(288, 10, LinearFunction(), seed=3),
], MSEFunction(), ExperimentLearningRateFunction())
```

This experiment is intentionally small because the implementation prioritizes transparency over numerical performance. The goal of the run was to verify the correctness of data loading, tensor-shape transitions, convolutional forward and backward passes, and figure generation.

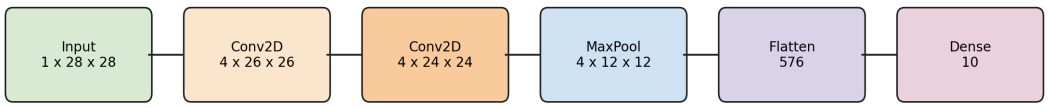

Figure 2: CNN architecture used for the MNIST sanity-check experiment.

5. Results

The executed experiment used 80 training samples and 40 test samples from the local MNIST PNG mirror, with four epochs of training. The purpose of this run was not to maximize accuracy, but to validate that the newly integrated convolution layer, pooling path, dataset loading, and metrics/figure generation operate end to end without manual intervention.

Figure 3 shows the evolution of training loss and test accuracy across epochs. The final training loss reached 0.0903. Final training accuracy was 7.5%, and final test accuracy was 2.5%. These values are obviously far below the performance expected from mature CNN implementations on MNIST. However,

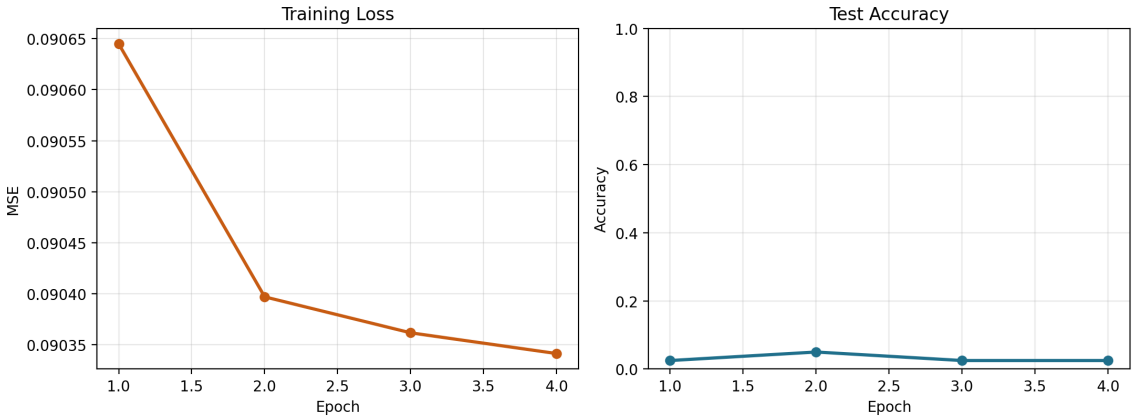

Figure 3: Training loss and test accuracy as functions of epoch for the pure Python CNN run.

they still provide a useful engineering result: the library successfully performed convolutional training on real image data, and the current optimization limitations became measurable rather than theoretical.

The confusion matrix in Figure 4 confirms that the present model configuration does not yet separate digit classes reliably. This outcome is consistent with the simplified optimization stack, the very small training subset, and the use of a linear output layer with mean squared error instead of a more appropriate softmax-cross-entropy pipeline. The low score should therefore be interpreted as a baseline validation of software functionality, not as a competitive benchmark.

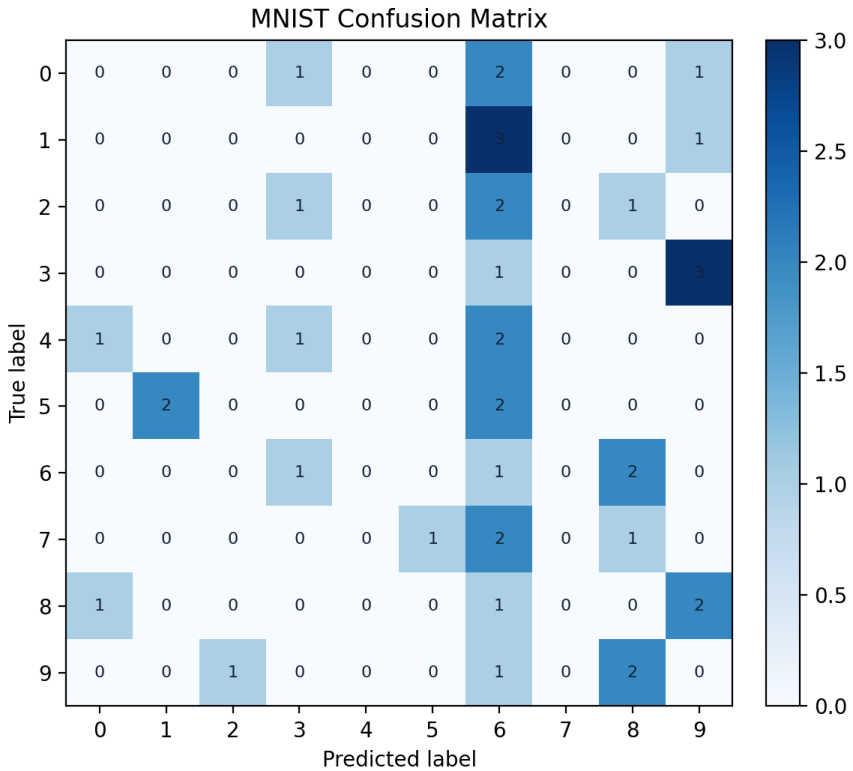

Figure 4: Confusion matrix for the MNIST test subset used in the experiment.

6. Discussion

The main outcome of the work is software validation rather than state-of-the-art recognition quality. From an implementation perspective, the project now supports the official MNIST binary formats, exposes a lazy loading path that is more RAM-friendly than eagerly duplicating the dataset, and runs a complete convolutional experiment that generates reproducible visual artifacts. These are meaningful improvements over an earlier draft state with an unfinished parser and missing figures.

At the same time, the empirical results make the current limitations explicit. The optimizer and loss setup are still rudimentary, the experiment uses a very small balanced subset because of pure Python execution cost, and the library lacks conveniences commonly found in modern deep-learning stacks, such as softmax output, cross-entropy loss, vectorized tensor kernels, and hardware acceleration. Future work should therefore focus on classification-oriented losses, better numerical stabilization, richer logging, and experiments on the full IDX dataset rather than a small PNG subset. Such improvements would make the library a stronger educational platform and a more convincing research demonstrator.

7. Conclusion

This paper documented a pure Python neural network library whose primary value lies in transparency, inspectability, and extensibility. The project was extended with an MNIST IDX parser, corrected dataset shuffling, improved convolutional compatibility, and a working CNN experiment on handwritten digits. Although the current convolutional benchmark remains far from production-level accuracy, the implementation now provides a concrete and reproducible foundation for further work on optimization, dataset scaling, and experimental methodology. For educational use, this explicit implementation remains valuable precisely because it reveals the details that optimized frameworks intentionally hide.

Acknowledgments

The author thanks the project mentor for feedback on the experimental scope, publication requirements, and draft preparation process.

References

[1] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, et al., Tensorflow: Large-scale machine learning on heterogeneous distributed systems, CoRR abs/1603.04467 (2016). URL: https://arxiv.org/abs/1603.04467.

[2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, Advances in Neural Information Processing Systems 32 (2019) 8024–8035.

[3] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, Nature 323 (1986) 533–536. doi:10.1038/323533a0.

[4] G. Cybenko, Approximation by superpositions of a sigmoidal function, Mathematics of Control, Signals, and Systems 2 (1989) 303–314. doi:10.1007/BF02551274.

[5] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, 2010, pp. 249–256.

[6] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86 (1998) 2278–2324. doi:10.1109/5.726791.

[7] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems 25, 2012, pp. 1097–1105.

[8] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, CoRR abs/1409.1556 (2015). URL: https://arxiv.org/abs/1409.1556.

[9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2015) 1–9. doi:10.1109/CVPR.2015.7298594.

[10] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2016) 770–778. doi:10.1109/CVPR.2016.90.

[11] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Advances in Neural Information Processing Systems 30 (2017) 5998–6008.

[13] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, International Conference on Learning Representations (2015). URL: https://arxiv.org/abs/1412.6980.

[14] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, Proceedings of the 32nd International Conference on Machine Learning (2015) 448–456.

[15] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, Proceedings of the IEEE International Conference on Computer Vision (2015) 1026–1034. doi:10.1109/ICCV.2015.123.